

PRODUCT REQUIREMENTS DOCUMENT

MediSight AI

Enterprise Healthcare Early Disease Risk Prediction & Clinical Decision Support Platform

Case Study Reference: ML-003 | Ezitech Engineering Framework (EEF)
Internship Deliverable — Ezitech Institute

Field	Detail
Document Type	Product Requirements Document (PRD)
Prepared By	Abdul Wahid — ML Engineer Intern
Team Size	2 ML Engineers
Duration	4 Weeks
Version	1.0
Status	Draft for Review

Table of Contents

1. Executive Summary
2. Problem Statement & Business Context
3. Goals, Objectives & Success Metrics
4. User Personas & Roles
5. Scope of Work (In-Scope / Out-of-Scope)
6. System Architecture Overview
7. Frontend Requirements
8. Backend Requirements (Supabase)
9. Authentication & Authorization
10. User Profile Management
11. AI Assistant (Conversational Clinical Copilot)
12. Machine Learning & Disease Prediction Engine
13. Explainable AI (XAI)
14. Clinical Dashboard & Reporting
15. Alerting & Notification System
16. MLOps & Model Lifecycle Management
17. Non-Functional Requirements
18. Additional Recommended Enhancements
19. Data Model (Database Schema)
20. API Design Overview
21. 4-Week Delivery Roadmap
22. Deliverables Checklist
23. Evaluation Criteria Mapping
24. Risks & Mitigations
25. Appendix

1. Executive Summary

MediSight AI is an enterprise-grade Clinical Decision Support Platform designed for a healthcare network operating 80 hospitals, 2,500 doctors, and over 20 million patient records. The platform ingests structured patient data (demographics, vitals, lab results, medication history) and applies machine learning models to estimate disease risk across six conditions — Diabetes, Heart Disease, Stroke, Kidney Disease, Liver Disease, and Hypertension — surfacing explainable, actionable insights to clinicians through a secure web dashboard.

This PRD translates the Ezitech ML-003 case study brief into a complete, buildable product specification, covering frontend UX, backend architecture on Supabase, authentication, an AI clinical assistant, explainable AI, MLOps, and non-functional requirements. It is written to be handed directly to a 2-person engineering team for a 4-week sprint, and to double as an internship deliverable demonstrating full-stack ML product thinking, not just model training.

Key outcomes this PRD enables:

- A clear, buildable spec covering frontend, backend, auth, AI assistant, and MLOps — nothing left ambiguous for engineering.
- A Supabase-first backend architecture (Postgres + Auth + Storage + Edge Functions + Realtime + pgvector) that keeps infra cost and complexity low for a 4-week case study.
- An explainability-first prediction UX so every risk score a doctor sees is accompanied by a plain-language reason.
- A defensible internship submission that maps directly onto the official evaluation rubric (Section 23).

2. Problem Statement & Business Context

The healthcare network's current workflow relies on manual chart review and clinical judgment alone to flag at-risk patients. This causes late disease detection, ICU overcrowding, high readmission rates, inconsistent patient prioritization, and rising treatment costs. There is no systematic, data-driven way to rank 20 million patients by risk or to surface the reasoning behind a risk estimate to a time-constrained clinician.

Current Challenges (from case brief)

- Late disease detection due to reactive, symptom-triggered assessment.
- ICU overcrowding from lack of early-warning signals.
- High readmission rates tied to weak post-discharge risk tracking.
- Manual, inconsistent patient prioritization across 80 hospitals.
- Limited predictive insight into population health trends.
- Increasing treatment costs from late-stage interventions.

Why now

Modern gradient-boosted models (XGBoost, LightGBM) combined with SHAP-based explainability have matured enough to be clinically trustworthy when paired with a transparent UI — making this the right moment to introduce ML-assisted triage without replacing clinical judgment.

3. Goals, Objectives & Success Metrics

Primary Objective

Develop a platform that analyzes patient records, lab results, vitals, and historical diagnoses to estimate disease risk, prioritize patients, and provide explainable, evidence-based recommendations to clinicians.

Product Goals

- Reduce time-to-identify a high-risk patient from a full chart review to a single dashboard glance.
- Give every prediction a transparent, doctor-readable explanation (no black-box scores).
- Give administrators visibility into model health (drift, retraining, accuracy over time).
- Provide a conversational AI assistant so clinicians can query risk context in natural language instead of navigating multiple screens.

Success Metrics

Metric	Target
Model AUC-ROC (per disease, baseline vs XGBoost/LightGBM)	≥ 0.80 on held-out test set
Prediction API latency (p95)	< 300 ms per patient
Dashboard load time (high-risk list, 10k patients)	< 2 s
Explainability coverage	100% of predictions show top-5 SHAP features
Alert delivery latency (critical risk detected → clinician notified)	< 60 s
Authentication success / MFA adoption for clinical roles	100% RLS-protected data access

4. User Personas & Roles

Persona	Goals	Key Screens
Attending Doctor	Quickly see which of their patients are high risk and why; act on alerts.	Dashboard, Patient Detail, Alerts, AI Assistant
Nurse / Care Coordinator	Monitor vitals trends and flag deteriorating patients.	Patient List, Patient Detail, Alerts
Hospital Administrator	See department-level and cross-hospital population health trends.	Analytics Dashboard, Department Statistics
Data Scientist / ML Engineer	Manage models, monitor drift, trigger retraining.	Admin Panel, Model Registry, Monitoring
Compliance / IT Admin	Manage user access, review audit logs.	Admin Panel, Audit Log, User Management

5. Scope of Work

In Scope

- Web application (responsive, desktop-first for clinical workstations, usable on tablets).
- Risk prediction for one disease at launch (recommend Diabetes or Heart Disease as the seed dataset — Pima Diabetes or UCI Heart Disease datasets are readily available), architected for expansion to all six conditions.
- Supabase-backed authentication, database, storage, and edge functions.
- Explainable AI (SHAP) surfaced in the UI, not just logged in the backend.
- Clinical dashboard with high-risk list, population trends, and department statistics.
- Rule-based + ML-triggered alerting system.
- AI conversational assistant scoped to a doctor's own patient panel.
- Basic MLOps: MLflow tracking, model registry, manual + scheduled retraining trigger, drift detection.

Out of Scope (for this 4-week cycle — noted as future work)

- Native mobile apps (responsive web only).
- Full HL7/FHIR EHR integration (interfaces designed to be FHIR-compatible later — see Section 18).
- Medical image analysis (imaging is a listed bonus challenge, not core).
- Real-time wearable device streaming (design hooks only).
- Multi-language localization beyond English.

6. System Architecture Overview

The platform follows a three-tier architecture: a Next.js frontend, a Supabase backend (Postgres + Auth + Storage + Realtime + Edge Functions) for all product data and auth, and a separate Python ML/FastAPI microservice for training, batch scoring, and SHAP computation. The two backends communicate over a secured internal API; Supabase Edge Functions act as the orchestration layer clients call into, which then invoke the FastAPI inference service.

High-Level Components

- **Frontend (Next.js)** — clinician-facing web app; calls Supabase client SDK directly for CRUD/auth, and calls Edge Functions for anything requiring the ML service or LLM.
- **Supabase Postgres** — system of record: patients, vitals, labs, predictions, alerts, users, audit logs.
- **Supabase Auth** — authentication, session/JWT issuance, MFA, RLS-enforced row-level access.
- **Supabase Storage** — avatars, exported PDF reports, model artifacts (or artifact metadata pointing to MLflow).
- **Supabase Edge Functions (Deno)** — thin orchestration layer: on-demand prediction requests, AI assistant proxy, alert dispatch, PDF export triggers.
- **ML/FastAPI Microservice** — Python service hosting XGBoost/LightGBM/Random Forest/Logistic Regression models, SHAP computation, batch scoring jobs; tracked via MLflow.
- **AI Assistant Layer** — LLM (Claude/OpenAI) + Retrieval over a patient's de-identified structured summary and SHAP output, exposed via an Edge Function so the frontend never holds an LLM API key.
- **MLflow + Model Registry** — experiment tracking, model versioning, staging/production promotion.

Data Flow (typical prediction request)

1. Clinician opens a patient detail page in the Next.js app.
2. Frontend calls a Supabase Edge Function (predict-risk) with the patient ID, authenticated by the user's JWT.
3. Edge Function verifies RLS-equivalent access (user is assigned to this patient/hospital), then calls the FastAPI inference service.
4. FastAPI loads the current production model from the Model Registry, engineers features from Postgres data, returns probability, severity level, and SHAP values.
5. Edge Function writes the prediction + SHAP payload to the predictions and explanations tables, and returns the result to the frontend.
6. If risk crosses the Critical threshold, the Edge Function inserts an alert row and triggers the notification channel (in-app + email).

7. Frontend Requirements

7.1 Tech Stack

- Framework: Next.js 14+ (App Router, React Server Components where suitable) with TypeScript.
- Styling: Tailwind CSS + shadcn/ui component primitives for a consistent, accessible design system.
- Data/state: TanStack Query (React Query) for server state/caching; Zustand for lightweight client/UI state (e.g., sidebar, theme).
- Charts: Recharts for trend lines, risk distributions, population heatmaps.
- Forms: React Hook Form + Zod for schema validation, shared with backend validation logic where possible.
- Supabase client: @supabase/supabase-js and @supabase/ssr for server-side auth/session handling in Next.js.
- Icons: lucide-react. Animations: Framer Motion (subtle, clinical-appropriate — no gratuitous motion).

7.2 Design Principles

- Clarity over decoration: this is a clinical tool used under time pressure — high information density, minimal chrome, no ambiguous icons without labels.
- Color is meaningful, not decorative: a strict, consistent risk-level palette (e.g., green/amber/orange/red for Low/Moderate/High/Critical) used identically across every screen, chart, and badge.
- Never hide the 'why': every risk score is always visually paired with an entry point to its explanation — never a bare number.
- Accessible by default: WCAG 2.1 AA contrast, full keyboard navigation, screen-reader labels on all charts and icons (critical for a clinical tool).
- Responsive: desktop-first (clinicians primarily use workstations), but fully usable on tablets at the bedside; mobile view supports alerts + patient lookup at minimum.
- Dark mode: supported system-wide via a theme toggle stored in user profile preferences (useful for night-shift clinicians).

7.3 Information Architecture / Screens

Screen	Purpose	Key UI Elements
Login / Sign Up	Authenticate clinicians and staff.	Email+password form, magic link option, SSO button (Google), 'Forgot password' link, error states.
Onboarding	Collect role, specialty, hospital/department on first login.	Multi-step form, role selector, hospital picker.
Main Dashboard	At-a-glance view of the clinician's panel.	High-risk patient list widget, alert feed, risk distribution donut chart, quick search.
Patient List	Browse/filter/search all assigned patients.	Sortable table, risk-level filter chips, search bar, pagination/virtualized scroll for large panels.
Patient Detail	Full clinical picture for one patient.	Header (demographics, current risk badge), vitals trend charts, lab results table, prediction card, SHAP explanation panel, medication list, alert history, notes.
Predictions / Risk Scoring	Deep dive into a specific disease prediction.	Probability + confidence gauges, severity badge, SHAP waterfall chart, clinical explanation text block, 'Run new prediction' action.

Screen	Purpose	Key UI Elements
Alerts Center	Central feed of all triggered alerts.	Filterable list (critical/vitals/lab/readmission), acknowledge/resolve actions, timestamped audit trail.
AI Assistant	Conversational Q&A over a patient panel.	Chat interface, suggested prompts, citation of source data points, patient-context switcher.
Analytics / Clinical Dashboard (population)	Department/hospital-level trends.	Disease distribution charts, risk heatmap by department, time-series population trend lines.
Reports	Export/print clinical or admin reports.	Report builder, PDF export, date range picker, saved report templates.
Admin Panel	User & model management (admin/data-scientist roles only).	User management table, role assignment, model registry view, drift monitoring charts, retraining trigger button, audit log viewer.
Profile & Settings	Manage personal account.	Editable profile form, avatar upload, password/MFA management, notification preferences, theme toggle.

7.4 Component Library (representative, not exhaustive)

- **RiskBadge** — color-coded Low/Moderate/High/Critical pill used consistently across list rows, cards, and headers.
- **PatientRiskCard** — probability, confidence, severity, and a 'View why' CTA into the SHAP panel.
- **ShapWaterfallChart** — top contributing features with directional (increases/decreases risk) bars.
- **VitalsTrendChart** — multi-series line chart (BP, heart rate, blood sugar) with abnormal-range shading.
- **AlertToast / AlertFeedItem** — real-time (Supabase Realtime subscription) alert surfacing.
- **AIChatPanel** — streaming chat bubble UI with source citations back to specific lab values or vitals.
- **DataTable** — shared virtualized table used by Patient List, Alerts, Admin Users, and Audit Log.
- **ProfileForm / AvatarUploader** — profile editing with Supabase Storage-backed avatar upload and crop.

7.5 UX Details Worth Calling Out

- Empty, loading, and error states are designed for every data view — not an afterthought (skeleton loaders, not blank screens).
- Optimistic UI for low-risk actions (e.g., acknowledging an alert), with rollback on failure.
- Global command/search (Cmd+K) to jump to any patient by name/MRN — clinicians should never need more than 2 clicks to reach a patient.
- Session timeout warning modal (clinical data compliance best practice) with one-click re-authentication.

8. Backend Requirements (Supabase)

Supabase is used as the primary backend-as-a-service layer: Postgres for data, Supabase Auth for identity, Supabase Storage for files, Supabase Realtime for live alerts, and Edge Functions (Deno/TypeScript) for orchestration logic that must not live in the client. Heavy ML compute (training, batch scoring, SHAP) lives in a separate Python/FastAPI service that Edge Functions call into.

8.1 Why Supabase Fits This Case Study

- Postgres gives relational integrity for patient/vitals/lab data plus native JSONB for flexible SHAP payloads.
- Row Level Security (RLS) maps naturally onto healthcare access rules (a doctor only sees their hospital/panel's patients).
- Built-in Auth removes the need to hand-roll session/JWT/password-reset logic in a 4-week timeline.
- pgvector extension supports the AI Assistant's retrieval layer without a separate vector database.
- Realtime subscriptions power the live Alerts Center with minimal custom infrastructure.

8.2 Core Supabase Services Used

Service	Usage in this Platform
Supabase Auth	Email/password + magic link + Google OAuth sign-in; JWT issuance; MFA (TOTP) for clinical roles.
Postgres (with RLS)	System of record for all clinical, user, and prediction data; RLS policies enforce hospital/role-scoped access.
Storage	Avatar images, generated PDF report exports, de-identified dataset snapshots.
Realtime	Live push of new alerts and prediction updates to subscribed dashboards.
Edge Functions	predict-risk, ai-assistant-chat, send-alert, export-report, retrain-trigger, admin-user-invite.
pgvector extension	Embeddings store for the AI Assistant's retrieval-augmented context.
Database Webhooks	Fire on new critical-risk prediction rows to invoke the alert Edge Function.

8.3 ML / FastAPI Microservice (outside Supabase)

- Python + FastAPI service exposing /predict, /explain, /retrain, /model-status endpoints, called only by Supabase Edge Functions (never directly by the browser).
- Model stack: Logistic Regression (baseline), Random Forest, XGBoost, LightGBM, with an ensemble/stacking option for the production model.
- SHAP TreeExplainer used for XGBoost/LightGBM/Random Forest; coefficients used directly for Logistic Regression explanations.
- MLflow for experiment tracking and the model registry (Staging → Production promotion workflow).

9. Authentication & Authorization

9.1 Sign-Up / Sign-In Methods (Supabase Auth)

- Email + password sign-up with email verification required before first login.
- Magic link (passwordless) sign-in as an alternative for lower-friction access.
- Google OAuth SSO for hospital-managed Google Workspace accounts.
- Multi-Factor Authentication (TOTP via authenticator app) — required for Admin and Doctor roles handling PHI.
- Forgot-password flow via Supabase Auth's built-in recovery email + secure reset link.
- Session management via Supabase's JWT + refresh token, short-lived access tokens, auto-refresh in the Next.js middleware.

9.2 Roles & Row Level Security

Roles are stored on a profiles table (extending auth.users) and enforced through Postgres RLS policies rather than only in application code, so access control holds even if an Edge Function or client has a bug.

Role	Access Level
super_admin	Full access across all hospitals; user and model management.
hospital_admin	Full access scoped to their hospital_id; user management within hospital.
doctor	Read/write access to patients on their assigned panel; read-only on department analytics.
nurse	Read access to assigned patients' vitals/alerts; can acknowledge alerts, cannot edit predictions.
data_scientist	Access to model registry, drift monitoring, retraining triggers; de-identified data only by default.

9.3 Example RLS Policy Pattern

Illustrative (not literal SQL to be pasted verbatim — engineering should adapt per final schema):

```
create policy "doctors_view_assigned_patients" on patients for select using (
  exists (
    select 1 from doctor_patient_assignments a
    where a.patient_id = patients.id
    and a.doctor_id = auth.uid()
  )
);
```

9.4 Audit Logging

Every read/write of PHI-relevant tables (patients, predictions, lab_results) is logged to an audit_logs table via Postgres triggers, capturing actor, action, table, row id, and timestamp — required for regulatory-friendly logging called out in the original brief.

10. User Profile Management

A dedicated Profile & Settings screen lets every user manage their own account without admin intervention, backed directly by Supabase Auth + a profiles table + Storage.

10.1 Editable Profile Fields

- Full name, display name, professional title (e.g., MD, RN), specialty/department.
- Hospital/branch assignment (view-only for non-admins; editable by admins).
- Avatar photo — upload/crop, stored in a private Supabase Storage bucket with signed URLs.
- Contact email (triggers re-verification) and phone number (for SMS alert opt-in).
- Password change and MFA enrollment/reset (via Supabase Auth admin API).
- Notification preferences: in-app, email, SMS toggles per alert severity.
- Theme preference (light/dark/system) and default dashboard view.
- Active sessions list with 'sign out of all other devices' action.

10.2 Account Lifecycle

- Self-service profile edits write directly to profiles via Supabase client with RLS ensuring users can only update their own row.
- Sensitive changes (email, role) require re-authentication and, for role changes, admin approval workflow.
- Account deactivation (soft delete) rather than hard delete, preserving audit trail integrity.

11. AI Assistant (Conversational Clinical Copilot)

The AI Assistant is an added, product-differentiating feature beyond the original case brief's minimum scope: a chat interface that lets clinicians ask natural-language questions about a patient's risk profile, trends, and explanation, and get grounded, cited answers — without replacing the core structured dashboard.

11.1 Capabilities

- Summarize a patient's current risk status and top contributing factors in plain language on request.
- Answer scoped questions such as 'Why is this patient's stroke risk high?' or 'How has their blood pressure trended over the last 6 months?'
- Draft a clinical note summary from structured data (LLM-based clinical report summarization — a bonus challenge from the brief).
- Surface population-level questions for admins, e.g., 'Which department has the most critical-risk patients this month?'
- Always cite the underlying data point(s) it used (lab value, vital, SHAP feature) — never present an unsourced clinical claim.

11.2 Architecture

- Frontend chat UI streams tokens from a Supabase Edge Function (ai-assistant-chat), which holds the LLM API key server-side — never exposed to the browser.
- Retrieval-Augmented Generation (RAG): the Edge Function fetches the patient's structured summary + latest SHAP explanation + relevant vitals/labs, embeds/query via pgvector where semantic search over notes is needed, and constructs a grounded prompt.
- The assistant is explicitly scoped: it can only access patients the requesting user's RLS permissions already allow, enforced by passing the user's JWT through to all data-fetching steps inside the Edge Function.
- Guardrails: the system prompt restricts the assistant from generating diagnoses or prescriptions outright — it is a decision-support summarizer, and every response includes a visible 'AI-generated — verify with clinical judgment' disclaimer.
- Conversation history stored per user/patient in an ai_chat_sessions / ai_messages table pair for continuity and audit.

11.3 Suggested Prompts (UI affordance)

- "Summarize this patient's risk in one paragraph."
- "What changed since their last visit?"
- "Which lab values are driving this prediction?"
- "Draft a discharge risk summary."

12. Machine Learning & Disease Prediction Engine

12.1 Patient Data Pipeline

Ingests and normalizes: demographics, medical history, vital signs, laboratory results, medication records, previous diagnoses, lifestyle factors, and family history, landing in Postgres staging tables before feature engineering.

12.2 Feature Engineering

- BMI (derived from height/weight).
- Blood pressure trend features (rolling mean/slope over recent readings).
- Blood sugar trend features (fasting/postprandial deltas).
- Heart rate variability.
- Cholesterol ratios (LDL/HDL, total/HDL).
- Kidney function scores (eGFR-derived).
- Composite risk indexes blending multiple vitals/labs.
- Medication compliance features (refill gaps, adherence proxy).

12.3 Models

Logistic Regression serves as the interpretable baseline; Random Forest, XGBoost, and LightGBM are evaluated as candidate production models with hyperparameter optimization (e.g., Optuna) and feature selection; an ensemble/stacked model is considered if it materially improves AUC over the best single model without hurting explainability.

12.4 Risk Scoring Output

Every prediction returns four elements as a package, never the probability alone: severity level (Low/Moderate/High/Critical), probability score, confidence score, and the SHAP-based explanation payload.

13. Explainable AI (XAI)

- SHAP values computed per prediction and stored alongside it (not recomputed on every page view) for both auditability and performance.
- Top Contributing Features shown as a signed waterfall chart — features pushing risk up in one color, down in another.
- Feature importance also aggregated at the model level for the Admin Panel (global explainability, useful for model validation and regulator conversations).
- Prediction confidence surfaced separately from probability — a high probability with low confidence (e.g., sparse recent labs) is flagged distinctly in the UI.
- A short auto-generated 'Clinical Explanation' sentence (e.g., 'Elevated fasting glucose and rising BMI trend are the largest contributors to this Diabetes risk score') is generated from the top SHAP features using a lightweight template or LLM pass, giving non-technical clinicians a plain-language reading, not just a chart.

14. Clinical Dashboard & Reporting

- High-Risk Patient List — sortable/filterable by disease, severity, department, last-updated.
- Disease Distribution — proportion of panel/population at each risk level, per disease.
- Population Health Trends — time-series of average risk scores and case counts over weeks/months.
- Department Statistics — per-department patient counts, average risk, alert volume.
- Risk Heatmaps — hospital x department grid colored by aggregate risk, for administrators.
- Report Export — PDF export of a patient's clinical summary or a department's population report, generated via an Edge Function + a PDF rendering library, stored in Supabase Storage with a shareable signed link.

15. Alerting & Notification System

15.1 Triggers

- Critical Risk Detected — a new prediction crosses the Critical threshold.
- Sudden Vital Changes — a vital reading deviates beyond a configurable delta from the patient's rolling baseline.
- Abnormal Lab Results — a lab value falls outside clinically defined reference ranges.
- High Readmission Risk — a derived readmission-risk feature crosses its threshold.

15.2 Delivery Channels

- In-app real-time toast + Alerts Center feed via Supabase Realtime.
- Email notification (via an Edge Function calling a transactional email provider, e.g., Resend/SendGrid).
- Optional SMS for Critical-only alerts, respecting each user's notification preferences from their profile.

15.3 Alert Lifecycle

New → Acknowledged → Resolved, each transition timestamped and attributed to a user for audit purposes; unacknowledged Critical alerts escalate (re-notify) after a configurable interval.

16. MLOps & Model Lifecycle Management

- MLflow Tracking — every training run logs parameters, metrics (AUC, precision/recall, F1), and artifacts.
- Model Registry — Staging/Production stage transitions with an audit trail of who promoted what and when.
- Dataset Versioning — snapshot/version the training dataset alongside each model version for reproducibility.
- Automated Retraining — scheduled retraining job (e.g., monthly) plus a manual 'Retrain now' trigger in the Admin Panel, both routed through the FastAPI service and logged to MLflow.
- Data Drift Detection — statistical drift checks (e.g., population stability index) on incoming feature distributions vs. training distribution, surfaced as a chart in the Admin Panel with alerting if drift exceeds threshold.
- Performance Monitoring — ongoing tracking of live prediction distributions and, where ground truth becomes available (e.g., confirmed diagnosis), realized accuracy over time.

17. Non-Functional Requirements

Category	Requirement
Security	TLS everywhere; encrypted at rest (Supabase default); RLS on every PHI table; secrets never in client code.
Explainability	No prediction is ever displayed without an accompanying explanation entry point.
Scalability	Stateless Edge Functions and FastAPI service, horizontally scalable; Postgres read replicas considered if analytics load grows.
Availability	Target 99.5% uptime for the clinical dashboard; graceful degradation if the ML service is unavailable (show last-known prediction with a staleness flag).
Logging & Compliance	Regulatory-friendly audit logging of all PHI access; logs immutable/append-only.
Latency	Prediction API p95 < 300ms; dashboard interactive in < 2s for a 10k-patient panel.
Accessibility	WCAG 2.1 AA across all clinician-facing screens.
Data Privacy	De-identification applied wherever data leaves the core Postgres boundary (e.g., to the AI Assistant's embeddings) unless explicitly required and authorized.

18. Additional Recommended Enhancements

Beyond the original case brief and the standard build, the following are recommended because they meaningfully increase the platform's realism, safety, and evaluation strength without significantly expanding the 4-week timeline:

- **Role-based onboarding wizard** — guided first-login flow that assigns role/hospital and explains the risk-color legend, reducing clinician training overhead.
- **Global command palette (Cmd+K)** — fast patient/feature search, a small UX detail that reads as production-grade polish.
- **FHIR-shaped data model** — even without a live EHR integration, shaping the patients/labs/vitals tables to loosely mirror FHIR resource fields (Patient, Observation, MedicationStatement) makes future EHR integration far cheaper.
- **Feedback loop on predictions** — a lightweight 'Was this prediction accurate?' clinician feedback control, logged for future model evaluation — demonstrates awareness of the human-in-the-loop ML lifecycle.
- **Synthetic/de-identified demo dataset** — since real 20M-record hospital data won't be available for an internship case study, generate a realistic synthetic dataset (e.g., extend the Pima Diabetes or UCI Heart Disease datasets with synthetic vitals/labs time series) and say so explicitly in the README — this is expected and evaluators will look for this awareness, not real PHI.
- **API rate limiting & caching** — basic rate limiting on Edge Functions and caching of expensive analytics queries (e.g., with a materialized view refreshed periodically) shows production awareness.
- **Automated testing** — unit tests for feature engineering and API contracts, plus a small integration test suite hitting the FastAPI /predict endpoint with fixture data.
- **CI/CD pipeline** — a simple GitHub Actions pipeline running lint/tests on push and deploying the frontend (e.g., Vercel) and Edge Functions (Supabase CLI) — strong signal for the 'MLOps' and 'Documentation' rubric lines.
- **Multi-tenancy awareness** — even at a single-hospital demo scale, keeping hospital_id on every table now avoids a costly re-architecture if the platform is later shown to multiple hospitals.
- **Accessibility & internationalization scaffolding** — using a strings/i18n file from day one, even if only English ships, costs little now and a lot to retrofit later.

19. Data Model (Database Schema)

Representative core tables (not exhaustive DDL — engineering to finalize column-level types/constraints):

Table	Key Columns	Notes
profiles	id (fk auth.users), full_name, role, hospital_id, department, avatar_url, notification_prefs	1:1 extension of Supabase auth.users.
hospitals	id, name, address, region	Supports multi-tenancy from day one.
patients	id, hospital_id, mrn, demographics (jsonb), family_history (jsonb)	FHIR-shaped Patient resource fields.
vitals	id, patient_id, type, value, unit, recorded_at	Time-series; FHIR-shaped Observation.
lab_results	id, patient_id, test_name, value, unit, reference_range, recorded_at	Time-series lab data.
medications	id, patient_id, name, dosage, start_date, end_date, adherence_score	Feeds compliance features.
diagnoses	id, patient_id, condition, diagnosed_at, source	Historical diagnosis record.
predictions	id, patient_id, disease, probability, confidence, severity, model_version, created_at	One row per prediction run.
explanations	id, prediction_id, shap_payload (jsonb), clinical_explanation (text)	1:1 with predictions.
doctor_patient_assignments	doctor_id, patient_id, hospital_id	Drives RLS scoping.
alerts	id, patient_id, type, severity, status, created_at, acknowledged_by, resolved_at	Alert lifecycle.
ai_chat_sessions / ai_messages	id, user_id, patient_id, role, content, created_at	AI Assistant conversation history.
model_registry	id, name, version, stage, metrics (jsonb), created_at	Mirrors MLflow registry state for UI display.
audit_logs	id, actor_id, action, table_name, row_id, created_at	Append-only compliance log.

20. API Design Overview

20.1 Supabase Edge Functions

Endpoint	Method	Purpose
/predict-risk	POST	Requests a fresh prediction for a patient/disease; proxies to FastAPI /predict.
/ai-assistant-chat	POST	Streams an AI Assistant response scoped to the requesting user's accessible patients.
/send-alert	POST (internal/webhook)	Triggered by a DB webhook on new critical predictions; dispatches notifications.
/export-report	POST	Generates a PDF report and returns a signed Storage URL.
/retrain-trigger	POST (admin only)	Kicks off a retraining job on the FastAPI service, logs to MLflow.
/admin-user-invite	POST (admin only)	Invites a new user via Supabase Auth admin API with a pre-assigned role.

20.2 FastAPI ML Service

Endpoint	Method	Purpose
/predict	POST	Runs feature engineering + inference for a given patient payload; returns probability, confidence, severity.
/explain	POST	Returns SHAP values for a given prediction/patient.
/retrain	POST	Triggers a training run against the current dataset; logs to MLflow.
/model-status	GET	Returns current production model version, metrics, and last training date.

21. 4-Week Delivery Roadmap

Week	Focus	Key Deliverables
Week 1	Foundations	Supabase project setup (Auth, schema, RLS); synthetic/de-identified dataset prepared; baseline Logistic Regression model; Next.js app scaffold with auth flows.
Week 2	Core ML + Dashboard	XGBoost/LightGBM models trained and tracked in MLflow; SHAP integration; Patient List, Patient Detail, and Prediction screens built and wired to real data.
Week 3	Differentiators	AI Assistant (RAG + Edge Function) implemented; Alerts system (Realtime + notifications); Admin Panel (model registry, drift monitoring) built.
Week 4	Polish, MLOps, Docs	Automated retraining + drift detection finalized; accessibility & responsive pass; testing + CI/CD; architecture diagram, deployment guide, README, evaluation report, and presentation prepared.

22. Deliverables Checklist

- Complete source code (frontend + Supabase project + FastAPI ML service).
- ML pipeline (training scripts, MLflow experiment logs).
- Feature engineering pipeline (documented, reusable module).
- Prediction API (FastAPI + Edge Function orchestration).
- Clinical dashboard (fully wired frontend).
- Model evaluation report (metrics per model/disease, chosen production model + rationale).
- Architecture diagram (system + data flow, matching Section 6).
- Deployment guide (Supabase project setup, environment variables, FastAPI hosting steps).
- README (setup, dataset provenance/synthetic-data disclosure, how to run locally).
- Technical presentation (slides covering problem, architecture, model results, demo).

23. Evaluation Criteria Mapping

Mapping this PRD's sections directly onto the official case-study rubric, to make sure nothing scored is left unaddressed:

Rubric Criteria	Weight	Addressed In
Feature Engineering	20%	Section 12.2
Model Performance	20%	Section 12.3, Section 3 (Success Metrics)
Explainability	20%	Section 13, Section 7.4 (ShapWaterfallChart)
MLOps	15%	Section 16
Clinical Dashboard	10%	Section 14, Section 7.3
Documentation	10%	Section 22 (Deliverables)
Presentation	5%	Section 22 (Technical Presentation)

24. Risks & Mitigations

Risk	Mitigation
No real hospital data available for a 2-engineer, 4-week internship project.	Use a well-documented synthetic/public dataset (e.g., Pima Diabetes, UCI Heart Disease) extended with synthetic time-series vitals/labs; disclose this explicitly in the README.
Scope creep across 6 diseases in 4 weeks.	Ship one disease end-to-end first (per brief's own guidance), architect the schema/UI to generalize, and treat additional diseases as a stretch goal.
AI Assistant hallucinating clinical claims.	Strict RAG grounding, visible AI-disclaimer, and restricting the system prompt from issuing diagnoses/prescriptions.
RLS misconfiguration exposing cross-hospital data.	Write explicit RLS policy tests as part of the automated test suite before demo.
Supabase Edge Function cold-starts affecting prediction latency.	Keep the FastAPI service warm/always-on for the demo; cache the last prediction per patient for instant display while a fresh one computes in the background.

25. Appendix

25.1 Suggested Public Datasets for Seed Model

- Pima Indians Diabetes Dataset (UCI/Kaggle) — for the Diabetes risk model.
- UCI Heart Disease Dataset (Cleveland) — for the Heart Disease risk model.
- Synthetic vitals/labs time series can be generated to simulate the trend-based features described in Section 12.2.

25.2 Glossary

- RLS — Row Level Security (Postgres feature enforcing per-row access control).
- SHAP — SHapley Additive exPlanations, a model explainability technique.
- RAG — Retrieval-Augmented Generation, grounding an LLM's answer in retrieved data.
- PHI — Protected Health Information.
- MRN — Medical Record Number.

25.3 Document History

Version	Date	Change
1.0	22 Jul 2026	Initial PRD draft prepared from Ezitech ML-003 case study brief.